Why 18 electrons were slow in our code, and why folx + slogdet fixes it

Stage 8 speed study, moire-nnvmc. 30 October 2026. Measurements on khabab-server, one RTX 3090 (GPU 0).

Written for a reader who knows a little calculus and has never heard of folx. Every number in this report was measured unless it is marked *estimate* or *interpretation*.

The short version

1. When we moved from 6 to 18 electrons, one training iteration of our attention network took **0.577 s**, against **0.33 s** for the authors' code, and 4096 walkers did not fit on the GPU at all. 82 % of our time went into one piece: the **local energy**, which needs second derivatives.

2. Our Stage 4 code computes those second derivatives the simple way: it builds the full table of 36 × 36 = 1,296 second derivatives and then keeps only the 36 on the diagonal. At 6 electrons the table has only 144 entries, so it did not matter.

3. The authors use a library called **folx** that computes only what is needed. When we tried folx in our code, it failed with `INTERNAL: No valid config found`. The reason is our **determinant step**: Stage 3 adds the determinants with plain `det`. folx has a special shortcut rule for `slogdet` but none for `det`, so for `det` it falls back to building a huge table, which at 18 electrons needs about 39 GiB for 1024 walkers. The card has 24 GiB.

4. Adding the determinants with `slogdet`, as the authors do, lets folx use its shortcut. The local energy then takes **0.130 s and 1.37 GiB** instead of 0.476 s and 16.9 GiB, which is the same as the authors' own code (0.1305 s, 1.37 GiB).

5. A whole training iteration now takes **0.224 s** at 1024 walkers (before: 0.577 s; authors: 0.33 s) and **0.79 s** at 4096 walkers, using 4.87 GiB (authors: 1.06 s, 16.6 GiB). So your friend was right: one RTX 3090 handles 18 electrons easily, once the second derivatives are computed the right way.

6. Why our code was faster at 6 electrons: there, the local energy was small for everyone, and what mattered was the rest of the iteration. Our rest took about 0.04 s; the authors' took about 0.12 s. At 18 electrons our slow local energy outgrew that advantage.

7. The price: `slogdet` returns NaN if one determinant is *exactly* zero, which is why Stage 3 chose `det` in the first place (Section 8 shows an example). The authors, FermiNet and Psiformer all accept this risk. We will keep both versions and test that they agree.

1. Words used in this report

word	meaning	example
walker	One guess of where all the electrons are: an (x, y) position for each electron.	18 electrons → 36 numbers per walker.
batch	How many walkers are processed together in one iteration. A bigger batch gives a less noisy energy but needs more time and memory.	1024 (the authors' released settings) or 4096 (paper Table II).
splitting the batch	Giving part of the walkers to each GPU. Each GPU does its share, and the results are combined.	4096 walkers on 2 GPUs = 2048 per GPU.
iteration	One training step: move the walkers, compute the energy at each walker, update the network.	Table II asks for 150,000 iterations.
local energy	$E_{loc} = (H\Psi)/\Psi$ at one walker. Its average over the walkers is the energy we minimise.	Kinetic + moiré potential + Coulomb.
Laplacian	The sum of the second derivatives of Ψ, one for each coordinate. The kinetic energy needs it.	36 second derivatives for 18 electrons.
Hessian	The full table of all second derivatives $\partial^2 f/\partial x_i \partial x_j$, including the mixed ones.	36 × 36 = 1,296 entries for 18 electrons.

word	meaning	example
determinant	The antisymmetric part of the wave function. Our networks add 4 determinants.	4 matrices of size 18 × 18 for 18 electrons.
det, slogdet	Two ways to compute a determinant in JAX: det gives the number itself; slogdet gives its sign and the logarithm of its size.	det = −0.002 ↔ slogdet = (sign −1, log 0.002 = −6.21).
folx	A library that computes a Laplacian in one forward pass, without building the Hessian ('forward Laplacian'). The authors use it.	Version 0.2.19 in our environment.
scratch memory	Temporary GPU memory a calculation needs while it runs.	An RTX 3090 has 24 GiB.

2. What one training iteration does, and where the time goes

Every iteration does three things: (a) **move** each walker 30 times with the Metropolis rule, which needs only log|Ψ|; (b) compute the **local energy** at every walker, which needs the Laplacian; (c) update the network with **KFAC**. We timed each piece on its own (attention network, released sizes, 1024 walkers):

piece	6 electrons, ours	18 electrons, ours	18 electrons, authors
log|Ψ| once (for reference)	0.001 s	0.0028 s, 0.07 GiB	0.0027 s, 0.07 GiB
(a) 30 Metropolis moves	0.023 s	0.071 s	not timed separately
(b) local energy	0.049 s, 1.93 GiB	**0.476 s, 16.9 GiB**	0.1305 s, 1.37 GiB
(c) KFAC update	0.020 s	0.034 s, 0.58 GiB	not timed separately
whole iteration	0.087 s	0.577 s	0.33 s

At 18 electrons the local energy is 0.476 s of our 0.577 s. Everything else together is about 0.1 s. So the whole difference from the authors' code is in one piece, and the rest of this report is about that piece.

3. Two ways to compute a Laplacian

A small example. Take f(x, y) = x²y. Its four second derivatives are ∂²f/∂x² = 2y, ∂²f/∂y² = 0 and the two mixed ones, ∂²f/∂x∂y = ∂²f/∂y∂x = 2x. The Hessian is the table [[2y, 2x], [2x, 0]]. The Laplacian is only the diagonal: 2y + 0 = 2y. The mixed derivatives are never used. With 2 coordinates you compute 4 numbers and use 2. With 36 coordinates you compute 1,296 and use 36 (Figure 1).

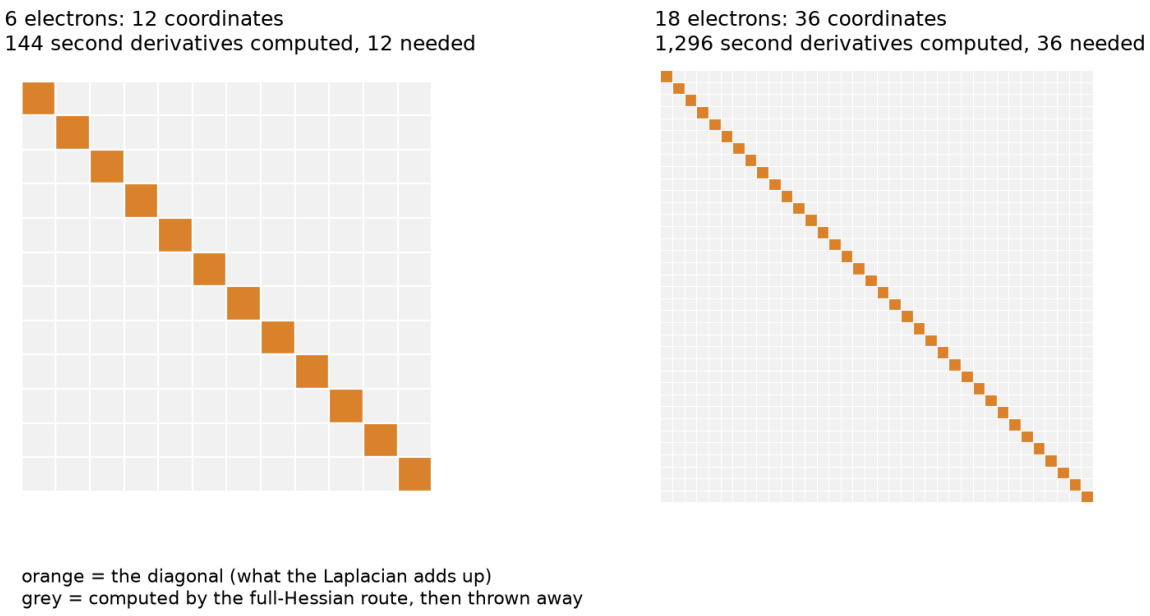


Figure 1. The Laplacian adds up only the diagonal of the Hessian (orange). Going from 6 to 18 electrons, the needed part grows 3 times (12 → 36) but the full table grows 9 times (144 → 1,296).

Route 1: the full Hessian (our Stage 4). Stage 4's `kinetic_energy` calls `jax.hessian`, once for log|Ψ| and once for the phase. JAX does this in two passes. First it records the whole network calculation so that it can run it backwards (that gives the 36 first derivatives). Then it repeats this forward-and-backward calculation in 36 directions at once, keeping the intermediate results of all 36 directions in memory together. Its docstring says so honestly: "*fast enough for a few electrons*". Measured cost of the local energy: 0.049 s and 1.93 GiB at 6 electrons; 0.476 s and 16.9 GiB at 18 electrons. At 4096 walkers it asked for a single 52 GB block of memory and stopped.

Route 2: the forward Laplacian (folx). folx goes through the network once, forwards only. For every intermediate number u it carries three things: the value u, its 36 first derivatives ∇u, and one running total, the Laplacian Δu. It never builds the 36 × 36 table and throws intermediate results away as soon as they are used. For this to work, folx needs a **rule** for each kind of operation, saying how (u, ∇u, Δu) become (y, ∇y, Δy). Example, for y = exp(u):

```
y  = exp(u)
∇y = exp(u) · ∇u
Δy = exp(u) · (Δu + |∇u|²)        ← no table needed, only u, ∇u and Δu
```

folx has such rules for adding, multiplying, exp, matrix products and most other building blocks of a network. The question is what happens when it meets an operation for which it has *no* rule.

4. Why folx + our Stage 3 determinant failed

What Stage 3 does. Our `sum_of_determinants` (Stage 3, `slaternet.py`) first rescales the 4 matrices so nothing overflows, and then adds `jnp.linalg.det` of each. We chose plain det on purpose: its derivative stays finite even when a matrix is singular (Section 8). That was a good choice for the Hessian route. It is a bad one for folx.

What folx knows. We read folx's own source code (version 0.2.19). Its list of special rules (`folx/wrapped_functions.py`) contains an entry `'slogdet'`, implemented in `folx/custom_hessian.py` as `slogdet_jac_hessian_jac`, with the comment "*For slogdet we know how to compute the determinant faster.*" There is no entry for det. We printed what JAX hands to folx for an 18 × 18 matrix (`jaxpr_probe.py`):

```
slogdet:  jit[name=slogdet]  →  folx looks up the name 'slogdet', finds its rule, done
det:      jit[name=det]      →  no rule for 'det', so folx looks inside and finds
            custom_jvp_call  →  det's own derivative rule: a closed box for folx
              (inside the box: jit[name=slogdet], exp, mul)
```

folx finds its rules by the *name* of the function (`eval_pjit` in `folx/interpreter.py`). Inside det it meets a `custom_jvp_call`: JAX gives det a hand-written derivative (the one that stays finite at singular matrices), and folx treats any such function as a closed box and applies its **general fallback** (`folx/hessian.py`, `general_jac_hessian_jac`). The irony: inside that box JAX computes the value of det *from a slogdet*. folx never sees it, because the box is sealed before folx gets there.

What the fallback does. It flattens all inputs of the box into one long list and, because that list is longer than the 36 coordinates, it computes the *full Hessian of the box with respect to every input* (`JHJ_via_hessian`). The inputs of our det box are all 4 matrices at once:

	6 electrons	**18 electrons**
matrix entries going into det (4 matrices of N × N)	4 × 6 × 6 = 144	4 × 18 × 18 = 1,296
second-derivative table per determinant, per walker	144 × 144 = 20,736	1,296 × 1,296 = 1,679,616
for all 4 determinants, per walker (the code's recipe)	82,944 numbers	6.7 million numbers
measured scratch memory per walker (det step alone)	0.50 MiB	39 MiB

folx does not know that determinant 1 depends only on matrix 1, so every determinant gets a table over all 1,296 entries. The size grows like N^4: from 6 to 18 electrons that is $3^4 = 81$ times. We measured 78 times (32 MiB → 2.45 GiB for 64 walkers). The measured memory (about 5

million 8-byte numbers per walker at 18 electrons) is of the same size as the recipe predicts. This is also why the problem never showed up before: at 6 electrons a whole training iteration with folx + plain det runs fine (0.061 s, 0.55 GiB).

The test that isolates it. To be sure the determinant is the culprit, we replaced the network with a tiny made-up function that produces 4 complex N × N matrices, and changed only how the determinants are added and differentiated (`det_alone.py`). All four routes give the same kinetic energy (33.1650, 33.1650, 33.1650, 33.1650 for 18 electrons, 64 walkers, float32), so they compute the same thing and differ only in cost:

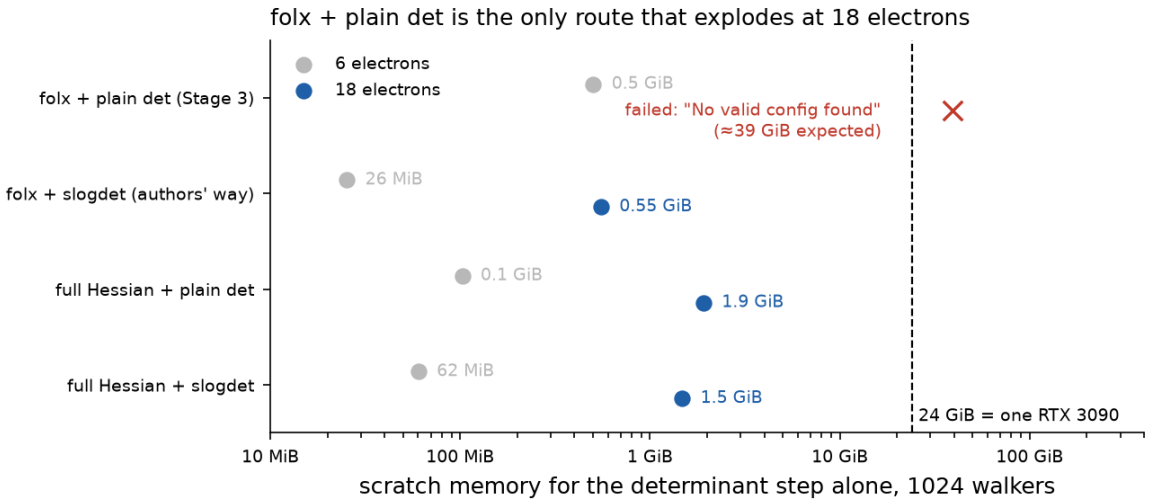


Figure 2. Scratch memory of the determinant step alone, 1024 walkers (compiler's memory analysis). Only folx + plain det explodes at 18 electrons, and it fails with exactly the error we saw in the full network. The ≈39 GiB is an *estimate*: 64 walkers need 2.45 GiB, and memory grows in proportion to the walkers (at 6 electrons we measured exactly 16 × from 64 to 1024).

The error message. `INTERNAL: No valid config found!` comes from XLA, the compiler under JAX, while it searches for a way to run one of the operations on the GPU. It does not say "out of memory". *Interpretation:* it gave up because no way of running the giant operation fits in the card; the same calculation at 64 walkers runs, and 1024 walkers would need about 39 GiB.

5. Everything we tried, and what happened

attempt (18 electrons)	result	why
Stage 4 full Hessian + Stage 3 det (our code as it was)	works; 0.476 s, 16.9 GiB at 1024; asks for 52 GB at 4096 and stops	the Hessian route stores every intermediate result for 36 directions
Stage 4 full Hessian + slogdet	0.432 s, 16.1 GiB: hardly better	the cost of this route is in the network, not in the determinant
folx (dense) + Stage 3 det, attention network	fails at 1024: "No valid config found"; asks for 58 GB at 4096	no folx rule for det → full Hessian over all 1,296 matrix entries
folx (dense) + Stage 3 det, SlaterNet	runs at 1024 (0.14 s, 9.84 GiB); fails at 4096	same fallback; SlaterNet happened to stay under 24 GiB at 1024 (we did not study why it needs less than the attention network)
folx with sparsity (threshold 0.6) + det	SlaterNet: 0.119 s, 9.84 GiB, no memory saved. Attention: folx crashes (IndexError) inside `jax.nn.dot_product_attention`	a folx limitation with that operation. The authors' own code comment says finite sparsity values crash, and they use 0. With an exact attention swapped in, sparse folx ran and agreed to 1e-12.
folx (dense) + slogdet	**0.1296 s, 1.37 GiB at 1024; 0.4636 s, 4.87 GiB at 4096**	folx's hand-written slogdet rule (next section)

6. Why folx + slogdet works so well

For log det A there is a compact formula. Its first derivative in a direction k is $\mathrm{tr}(A^{-1}\,\partial_k A)$, and the part of its second derivative that folx needs is $-\Sigma_k\ \mathrm{tr}(A^{-1}\,\partial_k A\ A^{-1}\,\partial_k A)$. Both need only **one** inverse A^{-1} (an 18 × 18 matrix) and the 36 first-derivative matrices $\partial_k A$, which folx already carries. folx's rule does exactly this: invert once, multiply, take a trace. Per determinant and walker that is about 18 × 18 × 36 = 11,664 numbers instead of 1.7 million. The same operations, arranged by someone who knew the formula.

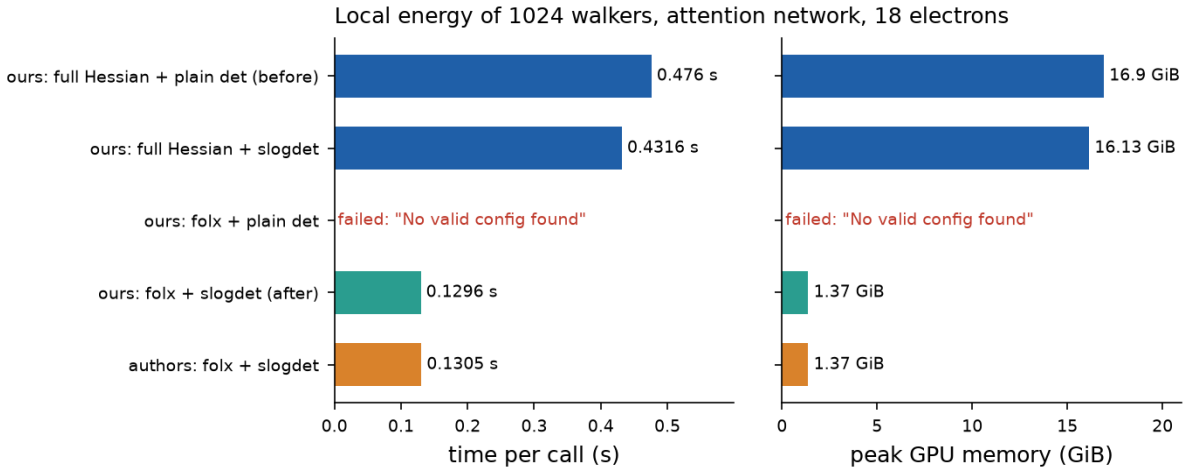


Figure 3. The local energy of the real attention network (released sizes, 18 electrons, 1024 walkers, one GPU). Switching only the determinant step to slogdet makes folx work, and the result matches the authors' code to within 1 %: 3.7 times faster and 12 times less memory than our old route.

Is it correct? In float64 on the CPU, the folx kinetic energy equals Stage 4's full-Hessian kinetic energy to 7.0e-13 (real part) and 4.2e-12 (imaginary part) for SlaterNet, and to 1.7e-12 and 3.3e-12 for the attention network (measure: largest difference divided by the largest value). With JAX's library attention the attention difference was 1.5e-4 and 1.1e-3, because `jax.nn.dot_product_attention` computes its softmax in float32 even in a float64 run. With an exact float64 attention swapped in it drops to 1e-12, so folx itself is exact. Our folx version keeps both the real and the imaginary part of the kinetic energy; the authors' code keeps only the real part.

7. Why our code was fast at 6 electrons and slow at 18, and the authors' the other way round

An analogy. Think of each iteration as cooking a meal. There is the chopping, which grows with the size of the meal (the local energy, which grows with the number of electrons), and there are fixed chores such as setting the table and washing up (moving walkers, the KFAC update, and the bookkeeping between steps). Our old kitchen had quick chores but a slow way of chopping. The authors' kitchen has a fast chopping machine but slower chores. For a small meal (6 electrons) the chores decide, and we were faster: 0.087 s against 0.141 s. For a big meal (18 electrons) the chopping decides, and we were slower: 0.577 s against 0.33 s. Now we have the chopping machine and keep our quick chores: 0.056 s at 6 electrons and 0.224 s at 18.

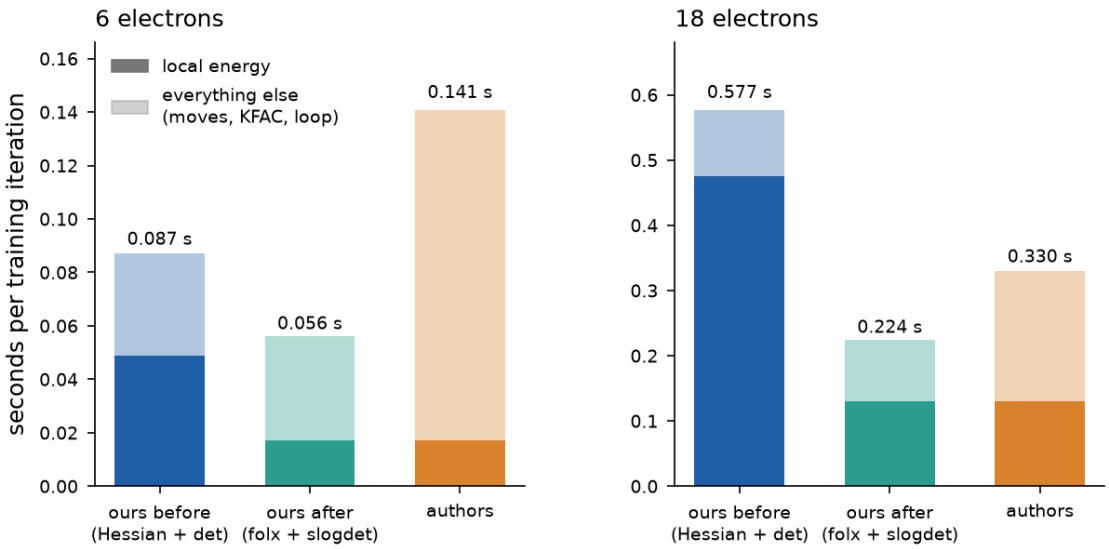


Figure 4. Seconds per training iteration, attention network, 1024 walkers, one GPU. Dark: the local energy, measured alone. Light: everything else (moves, KFAC, the training loop) = whole iteration minus local energy. The authors' 6-electron total (0.141 s) is their measured training speed, 7.1 iterations per second.

The numbers behind it. From 6 to 18 electrons our Hessian local energy grew 9.7 times (0.049 → 0.476 s), and the authors' folx local energy grew 7.7 times (0.017 → 0.1305 s). The rest of the iteration grew much less: ours from about 0.04 to 0.10 s, the authors' from about 0.12 to 0.20 s.

Where the authors' extra fixed time goes (*interpretation, not profiled*). At 6 electrons their GPU was busy only 14–28 % of the time, so most of each iteration the GPU was waiting. Their compiled step contains the same three pieces as ours (30 Metropolis moves, folx local energy, kfac_jax update) for a network with the same 103,232 parameters. After every step their training loop (`periodicwave/train.py`, lines 676–737) runs several small uncompiled JAX operations (statistics, a square root, a `jax.lax.cond` for the move width) and copies the energy and statistics to the CPU (`np.asarray`, every iteration because `stats_frequency = 1`). Each such operation makes Python wait for the GPU and the GPU wait for Python. *Estimate:* if their moves and KFAC update cost what ours do, about 0.08 s (6 electrons) to 0.1 s (18 electrons) of each iteration is spent this way. We did not time these operations one by one, so we cannot say how that time splits between them. Their NaN check would copy all parameters to the CPU, but it is switched off (`check_nan: False`).

8. The price of slogdet

Stage 3's docstring explains why we did not use slogdet: "*The derivative of slogdet is d log det A = tr(A^-1 dA), which is infinite when A is singular.*" When one of the 4 determinants is exactly zero while the others are not, Ψ itself is perfectly smooth, but the slogdet route multiplies that infinity by the vanishing determinant: 0 × ∞ = NaN. Plain det differentiates through the adjugate matrix and stays finite.

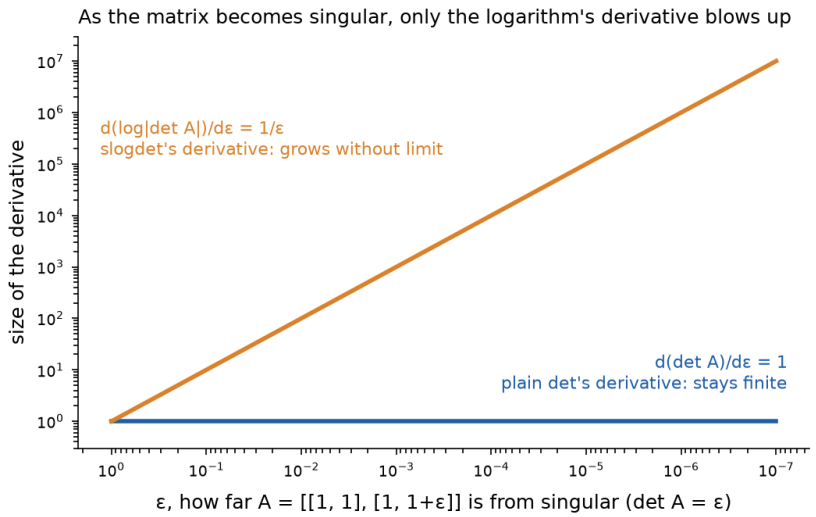


Figure 5. For A = [[1, 1], [1, 1+ε]], det A = ε. The derivative of det A is 1 for every ε, but the derivative of log|det A| is 1/ε, which grows without limit as A becomes singular.

A concrete test (`singular_example.py`, run with our real Stage 3 code). Ψ(ε) = det A(ε) + det B with B = diag(2, 1), so Ψ = ε + 2 and the exact answer is d log|Ψ|/dε = 1/(ε + 2):

ε	precision	exact	Stage 3 (det)	slogdet	exact 2nd deriv.	det 2nd	slogdet 2nd
1e-01	float64	0.4761905	0.4761905	0.4761905	-0.226757	-0.226757	-0.226757
1e-03	float64	0.4997501	0.4997501	0.4997501	-0.249750	-0.249750	-0.249750
1e-07	float64	0.5000000	0.5000000	0.5000000	-0.250000	-0.250000	-0.250000
0 (singular)	float64	0.5000000	0.5000000	NaN	-0.250000	-0.250000	NaN
1e-03	float32	0.4997501	0.4997501	0.4997500	-0.249750	-0.249750	-0.249725
0 (singular)	float32	0.5000000	0.5000000	NaN	-0.250000	-0.250000	NaN

So: at an exactly singular matrix, slogdet gives NaN and det gives the right answer. Close to singular, both work in float64; in float32 the slogdet second derivative loses digits a little earlier (-0.249725 instead of -0.249750 at $\varepsilon = 10^{-3}$).

Why we accept it anyway. Walkers are random real numbers, so landing exactly on a configuration where one determinant is exactly zero essentially never happens. The authors, FermiNet and Psiformer all add their determinants this way, with the same largest-value shift, and train normally. Our training loop checks every energy for NaN, but, as the review you pasted pointed out, it does not yet check the *updated* parameters before saving a checkpoint; that fix comes together with these changes. **What we will do:** keep Stage 3's det version unchanged, add the slogdet version next to it, test that both give the same log|Ψ| and the same local energy on ordinary configurations, and add a test that documents the NaN at an exactly singular matrix.

9. What this means for the 18-electron training

Wall-time *estimates* for one run on one GPU, from the measured seconds per iteration (5 iterations from random walkers, released network sizes; a real run adds checkpoints and logging, and paper Table II uses a slightly smaller attention network: 3 layers, 6 heads):

network, settings	before (Hessian + det)	after (folx + slogdet)	authors' code, 1 GPU
attention, 1024 walkers, 200,000 iterations	0.577 s/it → 32 h	0.224 s/it → 12.4 h	0.33 s/it → 18 h
attention, 4096 walkers, 150,000 iterations (Table II)	does not fit on the GPU	0.79 s/it → 33 h	1.06 s/it → 44 h
SlaterNet, 1024 walkers, 200,000 iterations	0.125 s/it → 6.9 h	0.057 s/it → 3.2 h	not measured
SlaterNet, 4096 walkers, 150,000 iterations	does not fit on the GPU	0.176 s/it → 7.3 h	not measured

All of these are for **one GPU**. The authors' code on two GPUs took 0.615 s per iteration at 4096 walkers. Our code does not yet split a batch across two GPUs; adding that is the next step after the folx and slogdet changes, and its speed is not measured yet. Running two different jobs, one per GPU, works already.

10. How the measurements were made

Hardware: khabab-server, NVIDIA RTX 3090 (24 GiB), driver 595.99.02. Every measurement in this study ran on **GPU 0 only**; GPU 1 stayed idle (its log shows 0 % use and 4 MiB). Software: Python 3.13, JAX 0.8.0, folx 0.2.19, kfac_jax 0.0.7, float32 with `jax_default_matmul_precision = float32`.

Pieces (`pieces.py`): each piece is a compiled function, called 6 times; the first call includes compilation and is dropped; the time is the mean of calls 2–6; peak memory is JAX's `peak_bytes_in_use` on GPU 0 for the whole process. Determinant step alone (`det_alone.py`): the same, plus the compiler's memory analysis (`temp_size_in_bytes`) before running. Full iteration (`full_iteration.py`): moves + local energy + KFAC, 6 iterations from random walkers, mean of the last 5. Authors' full iteration (earlier smoke tests): total time of 240 iterations minus total time of 40, divided by 200, memory with `XLA_PYTHON_CLIENT_PREALLOCATE=false`. The 18-electron piece numbers (pieces18.py) were measured a few hours before this study, with the same method on the same GPU; the 6-electron pieces, the determinant-step test and the new full iterations are from this study. GPU usage was logged every 2 s for both GPUs (`gpu_usage.csv`): GPU 0 peaked at 18.4 GiB, 100 % use, 56 °C.

Files (in `learn/stage08_own_code/speed_study/`): this report and its builder, the measurement scripts (pieces.py, det_alone.py, full_iteration.py, singular_example.py, jaxpr_probe.py, run_all.sh), `speed_results.jsonl` (one line per run), the logs of every run, the GPU usage log, and the earlier 18-electron logs.

Appendix A. How to repeat the study

```
cd ~/moire-nnvmc/learn/stage08_own_code/speed_study
./run_all.sh          # about 16 minutes, GPU 0 only
cat progress.txt      # which run is going
uv run --no-project --with matplotlib --with numpy python make_figures.py
uv run --no-project --with matplotlib --with reportlab python build_report.py
```

The project environment has no matplotlib or reportlab; `uv run --no-project --with ...` adds them for one command without changing the environment, as in the Stage 8 analysis folder.

Appendix B. Every run of today's study

run	seconds	peak memory	compiled scratch	note
det alone 6e folx det b64	0.00072	0.0821 GiB	0.0314 GiB	kinetic 3.15151
det alone 6e folx det b1024	0.00623	0.5518 GiB	0.5023 GiB	kinetic 2.95969
det alone 6e folx slogdet b64	0.00036	0.0321 GiB	0.0016 GiB	kinetic 3.15151
det alone 6e folx slogdet b1024	0.00048	0.045 GiB	0.0253 GiB	kinetic 2.95968
det alone 6e hessian det b64	0.0006	0.052 GiB	0.0079 GiB	kinetic 3.15151
det alone 6e hessian det b1024	0.00399	0.1047 GiB	0.1025 GiB	kinetic 2.95968
det alone 6e hessian slogdet b64	0.00046	0.0365 GiB	0.004 GiB	kinetic 3.15151
det alone 6e hessian slogdet b1024	0.00162	0.1033 GiB	0.0602 GiB	kinetic 2.95968
det alone 18e folx det b64	0.02116	2.4982 GiB	2.45 GiB	kinetic 33.16500
det alone 18e folx det b1024	—	—	—	JaxRuntimeError: INTERNAL: No valid config found!
det alone 18e folx slogdet b64	0.0005	0.0522 GiB	0.0381 GiB	kinetic 33.16499
det alone 18e folx slogdet b1024	0.00446	0.5518 GiB	0.5514 GiB	kinetic 29.31962
det alone 18e hessian det b64	0.00533	0.1406 GiB	0.1234 GiB	kinetic 33.16502
det alone 18e hessian det b1024	0.06409	1.9108 GiB	1.9104 GiB	kinetic 29.31963
det alone 18e hessian slogdet b64	0.00286	0.1027 GiB	0.0931 GiB	kinetic 33.16500
det alone 18e hessian slogdet b1024	0.03236	1.4765 GiB	1.4761 GiB	kinetic 29.31961
pieces 6e ours det logpsi	0.001	0.06 GiB	—	
pieces 6e ours det eloc hessian	0.0489	1.93 GiB	—	
pieces 6e ours det eloc folx	0.0223	0.55 GiB	—	
pieces 6e ours slogdet eloc hessian	0.0467	1.87 GiB	—	
pieces 6e ours slogdet eloc folx	0.017	0.14 GiB	—	
pieces 6e ours slogdet move	0.023	0.06 GiB	—	
pieces 6e ours slogdet kfac	0.0197	0.15 GiB	—	
pieces 6e authors logpsi	0.0011	0.06 GiB	—	
pieces 6e authors eloc folx	0.017	0.14 GiB	—	
full 6e attention folx det	0.061	0.55 GiB	—	
full 6e attention folx slogdet	0.056	0.15 GiB	—	
full 18e attention folx slogdet b1024	0.224	1.21 GiB	—	
full 18e attention folx slogdet b4096	0.79	4.87 GiB	—	
full 18e slaternet folx slogdet b1024	0.057	0.82 GiB	—	
full 18e slaternet folx slogdet b4096	0.176	2.76 GiB	—	

Seconds: per call (pieces, determinant step) or per iteration (full). All runs: GPU 0, float32, ε = 5, 1024 walkers unless the name says b64 or b4096.